

Windows Server, IIS, ARR, URL Rewrite, PM2, Node.js y React/Vite

Guia operativa para publicar frontend y backend sin exponer publicamente el puerto de Node.

Dato	Valor
Proyecto	CONEX
Version Node objetivo	20.19.1
Backend interno	http://127.0.0.1:3000
Frontend produccion	https://www.conexco.cl/free
API publica	https://www.conexco.cl/free/api/backendDocker/...
Fecha de documento	17 de agosto de 2026

Este documento resume los comandos usados durante la instalacion y configuracion, explica el sentido de cada paso y deja una metodologia reutilizable para desplegar nuevos proyectos frontend + backend en el mismo servidor.

Indice

Indice	2
1. Alcance del manual	4
Arquitectura aplicada	4
2. Versiones y requisitos	4
Comandos de verificacion	4
3. Instalacion base de Node y npm	4
4. Configuracion del backend CONEX	4
Ubicacion en servidor	4
Variables de entorno minimas	5
Instalar dependencias	5
Prueba manual del backend	5
5. Configuracion del frontend CONEX	5
Variables de produccion	5
Compilar frontend	5
Destino en servidor	5
6. IIS, URL Rewrite y ARR	5
Instalar ARR	5
Verificar ARR	6
Habilitar proxy ARR	6
web.config de CONEX	6
Prueba publica de la API	6
7. PM2 y servicio de Windows	6
Instalar PM2	6
Instalar integracion de servicio	6
Agregar backend a PM2	6
Verificar estado y logs	7

Reiniciar backend	7
Si el servicio queda detenido	7
8. Checklist de validacion	7
9. Guia para subir un nuevo proyecto	8
1. Definir rutas y puerto	8
2. Preparar backend	8
3. Crear .env.production del backend	8
4. Agregar backend a PM2	8
5. Preparar frontend	8
6. Publicar frontend en IIS	8
7. web.config del nuevo frontend	9
8. Validar proyecto nuevo	9
10. Recomendaciones para no afectar la aplicacion .NET	9
Patron recomendado	9

1. Alcance del manual

Este manual documenta la instalacion y configuracion aplicada para publicar CONEX en Windows Server. El despliegue contempla un frontend React/Vite publicado por IIS y un backend Node.js ejecutado con PM2 en un puerto local. El puerto de Node no queda expuesto directamente a internet.

La aplicacion .NET existente en IIS no debe ser modificada. Cada proyecto nuevo debe vivir en su propia carpeta, aplicacion virtual o sitio, con reglas URL Rewrite locales al proyecto.

Arquitectura aplicada

Usuario

```
-> https://www.conexco.cl/free
-> IIS sirve frontend estatico
-> https://www.conexco.cl/free/api/*
-> IIS URL Rewrite + ARR
-> http://127.0.0.1:3000/*
-> Backend Node / Express / PM2
-> SQL Server
```

2. Versiones y requisitos

Componente	Uso
Windows Server	Servidor donde ya existe IIS y una aplicacion .NET.
IIS	Sirve el frontend estatico y actua como entrada publica.
URL Rewrite	Permite definir reglas de reescritura en web.config.
ARR	Permite que IIS funcione como reverse proxy hacia Node.
Node.js 20.19.1	Version objetivo usada por el proyecto.
npm	Gestor de paquetes usado para instalar dependencias y compilar.
PM2	Administrador de procesos Node.
pm2-windows-service	Permite que PM2 arranque con Windows como servicio.
SQL Server	Base de datos usada por el backend mediante mssql / tedious.

Comandos de verificacion

```
node -v
npm -v
pm2 -v
```

3. Instalacion base de Node y npm

Como winget no estaba disponible en el servidor, se uso Chocolatey para instalar Node.

```
choco install nodejs-lts --version=20.19.1 -y
```

Si aparece error MSI 1603, normalmente indica un problema del entorno local, por ejemplo instalacion previa, reinicio pendiente o bloqueo del instalador. Revisar el log antes de repetir la instalacion.

```
Get-Content "C:\ProgramData\chocolatey\logs\chocolatey.log" -Tail 100
```

4. Configuracion del backend CONEX

Ubicacion en servidor

C:\inetpub\wwwroot\conex\backendConex

Variables de entorno minimas

```
NODE_ENV=production
PORT=3000
```

```
DB_SERVER=localhost
DB_PORT=1433
DB_INSTANCE=
DB_DATABASE=CONEX_MIGRACION
DB_USER=TU_USUARIO
DB_PASSWORD=TU_PASSWORD
```

```
DB_ENCRYPT=false
DB_TRUST_SERVER_CERTIFICATE=true
```

```
CORS_ORIGINS=https://www.conexco.cl
```

No se deben hardcodear servidores, usuarios, claves, base de datos ni puertos dentro del código. Las credenciales deben permanecer fuera de Git en archivos .env.

Instalar dependencias

```
cd C:\inetpub\wwwroot\conex\backendConex
npm install --omit=dev
```

Prueba manual del backend

```
$env:NODE_ENV="production"
node index.js
```

En otra consola PowerShell:

```
Invoke-WebRequest http://127.0.0.1:3000/ping -UseBasicParsing -TimeoutSec 10
Invoke-WebRequest http://127.0.0.1:3000/backendDocker/test -UseBasicParsing -TimeoutSec 10
```

5. Configuración del frontend CONEX

Variables de producción

```
VITE_APP_BASE_NAME=/free
VITE_API_URL=/free/api
```

Compilar frontend

```
npm install
npm run build
```

Destino en servidor

```
C:\inetpub\wwwroot\conex\frontConex
```

Se sube al servidor el contenido generado dentro de la carpeta dist.

6. IIS, URL Rewrite y ARR

Instalar ARR

```
$downloadDir = "C:\Temp\IISModules"
New-Item -ItemType Directory -Force $downloadDir
```

```
$arrMsi = "$downloadDir\requestRouter_amd64.msi"
```

```
Invoke-WebRequest `
  -Uri "https://download.microsoft.com/download/e/9/8/e9849d6a-020e-47e4-9fd0-a023e99b54eb/
requestRouter_amd64.msi" `
  -OutFile $arrMsi
```

```
Start-Process msixexec.exe `
  -ArgumentList @("/i", $arrMsi, "/qn", "/norestart", "/l*v", "$downloadDir\arr-install.log") `
  -Wait
```

Verificar ARR

```
& "$env:windir\system32\inetsrv\appcmd.exe" list config -section:system.webServer/proxy
```

Habilitar proxy ARR

```
& "$env:windir\system32\inetsrv\appcmd.exe" set config `
-section:system.webServer/proxy `
-enabled:"True" `
/preserveHostHeader:"True" `
/commit:apphost
```

ARR habilitado globalmente no redirige trafico por si solo. La redireccion concreta vive en el web.config del frontend, por eso no debería afectar la aplicacion .NET existente si no se modifica su configuracion.

web.config de CONEX

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <system.webServer>
    <rewrite>
      <rules>
        <rule name="CONEX_FRONT_API_PROXY" stopProcessing="true">
          <match url="^api/(.*)$" />
          <action type="Rewrite" url="http://127.0.0.1:3000/{R:1}" />
        </rule>

        <rule name="CONEX_FRONT_SPA" stopProcessing="true">
          <match url=".*" />
          <conditions logicalGrouping="MatchAll">
            <add input="{REQUEST_FILENAME}" matchType="IsFile" negate="true" />
            <add input="{REQUEST_FILENAME}" matchType="IsDirectory" negate="true" />
          </conditions>
          <action type="Rewrite" url="/free/index.html" />
        </rule>
      </rules>
    </rewrite>

    <staticContent>
      <mimeMap fileExtension=".webmanifest" mimeType="application/manifest+json" />
    </staticContent>
  </system.webServer>
</configuration>
```

Prueba publica de la API

```
Invoke-WebRequest https://www.conexco.cl/free/api/backendDocker/test -UseBasicParsing -
TimeoutSec 15
```

7. PM2 y servicio de Windows

Instalar PM2

```
npm install -g pm2
pm2 --version
```

Instalar integracion de servicio

```
npm install -g pm2-windows-service
& "$env:APPDATA\npm\pm2-service-install.cmd"
```

Agregar backend a PM2

```
cd C:\inetpub\wwwroot\conex\backendConex

pm2 start index.js `
  --name conex-backend `
  --cwd "C:\inetpub\wwwroot\conex\backendConex" `
  --env production

pm2 save
```

Verificar estado y logs

```
pm2 status
pm2 describe conex-backend
pm2 logs conex-backend --lines 100
Get-Service "pm2.exe"
```

Reiniciar backend

```
pm2 restart conex-backend --update-env
pm2 save
```

Si el servicio queda detenido

```
Get-EventLog -LogName Application -Newest 30 | Where-Object {
    $_.Source -like "*PM2*" -or $_.Message -like "*pm2*"
} | Select-Object TimeGenerated, Source, EntryType, Message
```

```
sc.exe qc "pm2.exe"
```

Si el servicio apunta a otro usuario, por ejemplo C:\Users\APAdmin\..., y ahora se esta trabajando con Administrator, eliminar el servicio viejo y reinstalarlo desde la cuenta correcta.

```
Stop-Service "pm2.exe" -ErrorAction SilentlyContinue
sc.exe delete "pm2.exe"
```

```
npm install -g pm2
npm install -g pm2-windows-service
& "$env:APPDATA\npm\pm2-service-install.cmd"
```

8. Checklist de validacion

- Node y npm responden con la version esperada.
- PM2 muestra conex-backend en estado online.
- El servicio pm2.exe esta configurado y puede iniciar con Windows.
- El puerto 3000 escucha solo de forma interna.
- La API responde en `http://127.0.0.1:3000/backendDocker/test`.
- La API responde publicamente a traves de IIS en `/free/api/backendDocker/test`.
- El frontend carga desde `/free`.
- El login llama a `/free/api/backendDocker/seguridad/login`.
- Las variables `.env` no estan versionadas en Git.

```
node -v
npm -v
pm2 -v
pm2 status
Get-Service "pm2.exe"
Get-NetTCPConnection -LocalPort 3000 -State Listen
Invoke-WebRequest http://127.0.0.1:3000/backendDocker/test -UseBasicParsing
Invoke-WebRequest https://www.conexco.cl/free/api/backendDocker/test -UseBasicParsing
```

9. Guía para subir un nuevo proyecto

1. Definir rutas y puerto

Elemento	Ejemplo recomendado
Frontend	C:\inetpub\wwwroot\nuevoProyecto\front
Backend	C:\inetpub\wwwroot\nuevoProyecto\backend
Ruta publica	https://www.tudominio.cl/nuevo
API publica	https://www.tudominio.cl/nuevo/api/...
Puerto Node	3001
Proceso PM2	nuevo-backend

2. Preparar backend

```
New-Item -ItemType Directory -Force "C:\inetpub\wwwroot\nuevoProyecto\backend"
cd C:\inetpub\wwwroot\nuevoProyecto\backend
npm install --omit=dev
```

3. Crear .env.production del backend

```
NODE_ENV=production
PORT=3001
```

```
DB_SERVER=localhost
DB_PORT=1433
DB_INSTANCE=
DB_DATABASE=NOMBRE_BD
DB_USER=USUARIO_BD
DB_PASSWORD=PASSWORD_BD
```

```
DB_ENCRYPT=false
DB_TRUST_SERVER_CERTIFICATE=true
```

```
CORS_ORIGINS=https://www.tudominio.cl
```

4. Agregar backend a PM2

```
cd C:\inetpub\wwwroot\nuevoProyecto\backend
```

```
pm2 start index.js `
  --name nuevo-backend `
  --cwd "C:\inetpub\wwwroot\nuevoProyecto\backend" `
  --env production
```

```
pm2 save
pm2 status
```

5. Preparar frontend

```
VITE_APP_BASE_NAME=/nuevo
VITE_API_URL=/nuevo/api
```

```
npm install
npm run build
```

6. Publicar frontend en IIS

```
New-Item -ItemType Directory -Force "C:\inetpub\wwwroot\nuevoProyecto\front"
```

Copiar el contenido de dist dentro de C:\inetpub\wwwroot\nuevoProyecto\front.

7. web.config del nuevo frontend

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <system.webServer>
    <rewrite>
      <rules>
        <rule name="NUEVO_API_PROXY" stopProcessing="true">
          <match url="^api/(.*)$" />
          <action type="Rewrite" url="http://127.0.0.1:3001/{R:1}" />
        </rule>

        <rule name="NUEVO_FRONT_SPA" stopProcessing="true">
          <match url=".*" />
          <conditions logicalGrouping="MatchAll">
            <add input="{REQUEST_FILENAME}" matchType="IsFile" negate="true" />
            <add input="{REQUEST_FILENAME}" matchType="IsDirectory" negate="true" />
          </conditions>
          <action type="Rewrite" url="/nuevo/index.html" />
        </rule>
      </rules>
    </rewrite>

    <staticContent>
      <mimeMap fileExtension=".webmanifest" mimeType="application/manifest+json" />
    </staticContent>
  </system.webServer>
</configuration>
```

8. Validar proyecto nuevo

```
Invoke-WebRequest http://127.0.0.1:3001/ping -UseBasicParsing -TimeoutSec 10
Invoke-WebRequest https://www.tudominio.cl/nuevo/api/ping -UseBasicParsing -TimeoutSec 15
pm2 logs nuevo-backend --lines 100
```

10. Recomendaciones para no afectar la aplicacion .NET

- No modificar el web.config raiz de la aplicacion .NET.
- Crear una carpeta independiente por cada frontend.
- Crear una aplicacion virtual o sitio independiente en IIS.
- Usar un puerto Node distinto por cada backend.
- Definir reglas URL Rewrite dentro del web.config del frontend nuevo.
- No exponer publicamente los puertos 3000, 3001 o similares.
- Guardar procesos PM2 con pm2 save despues de cada cambio.
- Mantener credenciales en .env.production y fuera de Git.

Patron recomendado

```
www.conexco.cl/free      -> frontend CONEX
www.conexco.cl/free/api/* -> Node puerto 3000

www.conexco.cl/nuevo     -> frontend nuevo
www.conexco.cl/nuevo/api/* -> Node puerto 3001
```